

Research Assistant: GROBID Setup Guide

Running the citation-extraction server with Docker

Shardul Mukim

2026-09-08

Welcome

GROBID is the component that reads a paper's bibliography properly. When you hand the Research Assistant a seed paper, something has to look at the reference list at the back and work out where one reference ends and the next begins — and for each one, pull out the title, the authors, the year, and the DOI.

GROBID does that job well. It runs as a small server on your own machine, and the Research Assistant talks to it over the network while it works.

You do **not** need to know programming to set this up. You need to install one program (Docker), paste two commands, and check that it worked.

Why this matters: The app still runs without GROBID — it falls back to pattern matching on the raw text. But that fallback is noticeably worse: the references it finds will **lack DOIs, authors and years**, so Agent 2 has much less to go on when deciding what to download, and many references that GROBID would have resolved will not download at all.

The app will not stop or show an error when this happens. It quietly does a worse job. That is exactly why everyone should have GROBID running before testing.

Part 1: Install Docker (Do this once)

Docker is the program that runs GROBID for you. Think of it as a way to run a piece of software that someone else has already set up and tested, without installing all its parts by hand.

Step 1: Download and install Docker

Pick the one that matches your computer:

- **Linux** — install **Docker Engine**: <https://docs.docker.com/engine/install/>
- **Mac** — install **Docker Desktop**: <https://docs.docker.com/desktop/install/mac-install/>
- **Windows** — install **Docker Desktop**: <https://docs.docker.com/desktop/install/windows-install/>

(On Mac and Windows, Docker Desktop is an app you launch like any other. On Linux, Docker runs as a background service.)

Step 2: Start Docker and check it is working

On **Mac or Windows**, open the **Docker Desktop** app and wait until it says “*Docker Desktop is running*”.

On **Linux**, start the service:

```
sudo systemctl start docker
```

Now open your **Terminal** and paste this to confirm Docker is ready:

```
docker info
```

If you see a long block of information, Docker is working. If you see an error, jump to **Part 5: Troubleshooting**.

Step 3 (Linux only): Fix “permission denied”

If `docker info` gave you a **permission denied** error, your user account is not yet allowed to talk to Docker. Fix it once with:

```
sudo usermod -aG docker $USER
```

Important: You must **log out and log back in** (or restart your computer) for this to take effect. Running the command alone is not enough.

Then run `docker info` again to confirm.

Step 4: Give Docker enough memory (Mac and Windows)

GROBID is a Java program and wants a reasonable amount of memory. In **Docker Desktop**, go to **Settings** → **Resources** and make sure **Memory** is set to at least **4 GB**. If it is lower, GROBID may start and then die a few seconds later.

(On Linux you can skip this — Docker uses the system’s memory directly.)

Part 2: Get the GROBID Server Running

Step 1: Download the GROBID image (one time)

Paste this and press **Enter**:

```
docker pull grobid/grobid:0.8.1
```

This downloads the prepared GROBID server. It is a few gigabytes, so it will take a while on a slow connection — but it only happens **once**. After this it is stored on your machine.

(This is the exact version the app expects. Please do not substitute a different tag unless you know why you are doing it.)

Step 2: Start the server

Paste this:

```
docker run --rm -d --name grobid -p 8070:8070 grobid/grobid:0.8.1
```

You will see a long string of letters and numbers. That is the server's ID, and it means the server has been launched.

Breaking that command down, in case you are curious:

Piece	What it does
<code>--rm</code>	Cleans the container up automatically when you stop it
<code>-d</code>	Runs it in the background, so you get your Terminal back
<code>--name grobid</code>	Gives it the name the app looks for
<code>-p 8070:8070</code>	Opens port 8070, which is where the app expects to find it

Be patient here. The command returns immediately, but GROBID is a Java service and needs roughly **30 seconds** to finish starting up. It is not ready the instant the command finishes.

Step 3: Check it is actually alive

Wait about half a minute, then paste:

```
curl http://localhost:8070/api/isalive
```

You want to see exactly this:

```
true
```

If you get **true**, you are done — GROBID is running.

If you get **Connection refused**, it is either still starting (wait another 30 seconds and try again) or it failed. See **Part 5**.

(You can also open <http://localhost:8070> in your browser — GROBID has its own small web page there.)

Step 4: Tell the app where it is

There is nothing to do. The app already looks for GROBID at `http://localhost:8070`, which is exactly where you just started it.

You only need to set anything if you are pointing at a GROBID running on *someone else's* machine, in which case:

```
export GROBID_SERVER=http://their-machine-address:8070
```

Part 3: Everyday Use

Starting GROBID again next time

Docker does not remember to start GROBID for you after a reboot. Each time you sit down to work, run the same command from Part 2:

```
docker run --rm -d --name grobid -p 8070:8070 grobid/grobid:0.8.1
```

Then start the app as usual. A good habit is to start GROBID first, make yourself a coffee, and launch the app after — by then the JVM has warmed up.

Using the app’s buttons instead

Once Docker is installed, you can drive GROBID from inside the app without touching the Terminal. In the sidebar under **Server & Services** you will find four small buttons (each carries an icon in the app):

- **Start GROBID** — launches the container and waits for it to answer
- **Stop** — stops the container
- **Restart** — stops and starts it again
- **Status** — asks the server whether it is awake

*(The status button renames itself depending on what the server is doing — you will see it as **Check Status** when stopped, **Refresh Status** while starting, and **Status** once running. It is the same button.)*

If you click **Start** and the app tells you “*GROBID container is running and initializing its JVM*”, that is normal and not an error. Wait half a minute and click **Refresh Status**.

One thing to know if you mix the two methods: The **Start GROBID** button creates a container that *stays on your system* after you stop it. The Terminal command in Part 2 uses `--rm`, which *deletes* it on stop.

So if you have used the button before, the Terminal command may fail with a **name conflict**. That is harmless — just start the existing one instead:

```
docker start grobid
```

Or remove it and start fresh:

```
docker rm grobid
```

Stopping GROBID

When you are finished for the day:

```
docker stop grobid
```

It is also fine to just leave it running. It sits idle using very little.

Part 4: How to Tell It Is Actually Being Used

Two quick checks, so nobody spends an afternoon testing the fallback by mistake:

1. **In the sidebar**, the **GROBID** indicator should be green, not red.
2. **In the Terminal running the app**, when a paper is processed you should see a line mentioning **references via grobid**. If you instead see **falling back to pattern matching or references via regex**, GROBID is not being reached.

If you see the fallback message, stop and fix GROBID before continuing your testing — otherwise you are testing a different, weaker system than everyone else.

Part 5: Troubleshooting Guide

What you see	What it means	How to fix it
Docker executable not found in PATH	Docker is not installed, or not installed for this user.	Go back to Part 1, Step 1. On Mac/Windows, confirm Docker Desktop is actually installed.
Docker daemon is not running	Docker is installed but switched off.	Open Docker Desktop , or on Linux run <code>sudo systemctl start docker</code> .
Docker socket permission denied	Your Linux user is not in the <code>docker</code> group.	Run <code>sudo usermod -aG docker \$USER</code> , then log out and back in .
GROBID image ... not found locally	The image was never downloaded.	Run <code>docker pull grobid/grobid:0.8.1</code> .
Port 8070 is already in use by another process	Something else has taken port 8070.	Find it with <code>docker ps</code> , then stop it — or set <code>GROBID_SERVER</code> to a different port.
Port 8070 is occupied by conflicting non-GROBID container	A different container is sitting on that port.	Stop that container, or move GROBID to another port.
Container conflict on 'grobid'	A stopped container with that name already exists.	Run <code>docker start grobid</code> , or <code>docker rm grobid</code> and try again.
GROBID failed to become ready within 25s	It is starting slowly, or it crashed.	Wait 30s and click Refresh Status . If still dead, run <code>docker logs grobid</code> and raise Docker's memory to 4 GB.
curl says Connection refused	The server is not up yet, or not running at all.	Wait 30 seconds and retry. Then check <code>docker ps</code> — is <code>grobid</code> listed?
Red GROBID dot, everything else fine	The app cannot reach the server.	Run <code>curl http://localhost:8070/api/isalive</code> . If that says <code>true</code> , restart the app.

If a container is misbehaving and you want to see why, this shows its own log:

```
docker logs grobid
```

Quick Reference

Everything on one card, for once you know what you are doing:

One time only

```
docker pull grobid/grobid:0.8.1
```

Every session

```
docker run --rm -d --name grobid -p 8070:8070 grobid/grobid:0.8.1
```

```
# Confirm it is awake (expect: true)
```

```
curl http://localhost:8070/api/isalive
```

```
# When you are done
```

```
docker stop grobid
```

Start the server, then put the kettle on. The Java Virtual Machine has never once been hurried by someone watching it.